
TwinTurbo

: A Bidirectional RHT Feeder IP for TurboQuant-based KV Cache Compressor

26 Summer Global Capstone Design



Yoonoh Hong, Seoyeong Byun, Hajin Choi
School of Electronic and Electrical Engineering, Sungkyunkwan University
E-mail: yoonoh27@g.skku.edu, michaela24@naver.com, oihjof@gmail.com

CONTENTS

01 Introduction

02 Background - TurboQuant

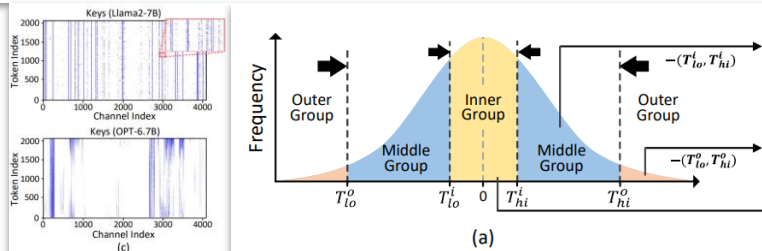
03 HW Implementation

04 Results

1. Introduction

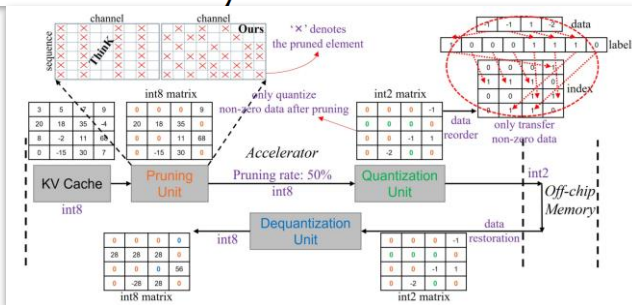
Introduction : Why TwinTurbo?

Oaken: Fast and Efficient LLM Serving with Online-Offline Hybrid KV Cache Quantization



"We select a ratio of 4%, 90%, and 6% for the outer, middle, and inner groups, respectively, throughout the entire evaluation"
Oaken [Kim et al., ISCA 2025]

Titanus: Enabling KV Cache Pruning and Quantization On-the-Fly for LLM Acceleration



Titanus [Chen et al., GLSVLSI 2025]

1 Limitation of Prior Work

: Prior HW Quantizers often **lack of mathematical guarantees**.

2 TurboQuant-based Quantization

: TurboQuant based-Quantization has **mathematical base of lower-bound** of error (TurboQuant has HLL implementation only)

3 Why we need "HW" RHT Quantizer?

: Unlike GPUs, edge devices are **difficult to allocate extra resources for quantization**.

Introduction : Why TwinTurbo?

TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate

Amir Zandieh
Google Research
zandieh@google.com

Majid Daliri
New York University
daliri.majid@nyu.edu

Majid Hadian
Google DeepMind
majidh@google.com

Vahab Mirrokni
Google Research
mirrokni@google.com

As demonstrated by our lower bounds, TURBOQUANT's MSE distortion is provably within a factor of at most $\frac{\sqrt{3}\pi}{2} \approx 2.7$ of the information-theoretical lower bound. Notably, for smaller bit-widths, this factor significantly decreases. For instance, at a bit-width of $b = 1$ TURBOQUANT achieves a distortion that is only a factor of approximately 1.45 away from the optimal which is also confirmed by our experimental results, indicating its efficiency in low-bit-width scenarios.

1 Limitation of Prior Work

: Prior HW Quantizers often **lack of mathematical guarantees**.

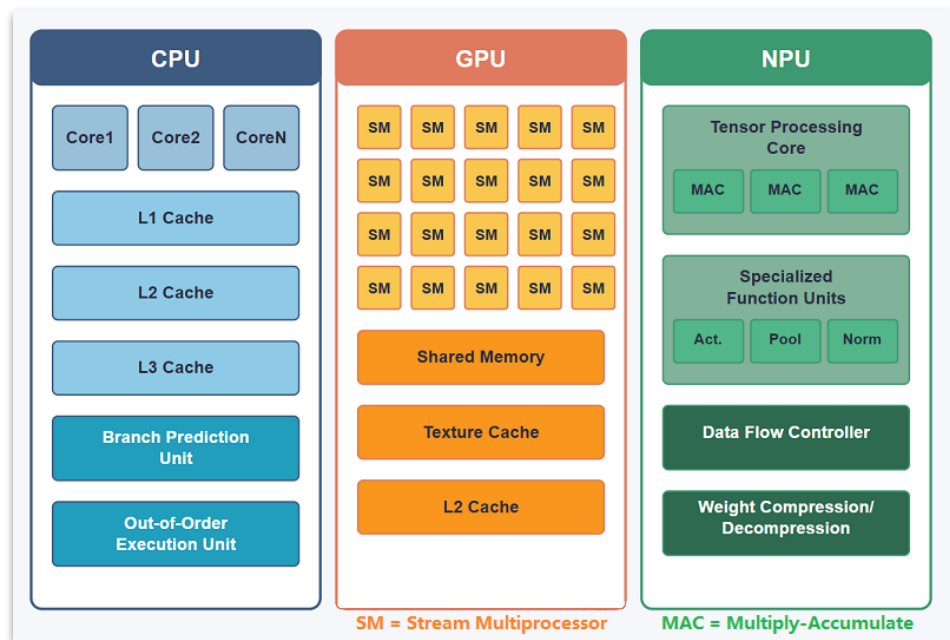
2 TurboQuant-based Quantization

: TurboQuant based-Quantization has **mathematical base of lower-bound** of error (TurboQuant has HLL implementation only)

3 Why we need “HW” RHT Quantizer?

: Unlike GPUs, edge devices are **difficult to allocate extra resources for quantization**.

Introduction : Why TwinTurbo?



1 **Limitation of Prior Work**
: Prior HW Quantizers often **lack of mathematical guarantees**.

2 **TurboQuant-based Quantization**
: TurboQuant based-Quantization has **mathematical base of lower-bound** of error (TurboQuant has HLL implementation only)

3 **Why we need “HW” RHT Quantizer?**
: Unlike GPUs, edge devices are **difficult to allocate extra resources for quantization**.

2. Background - TurboQuant

Algorithm1 : TURBOQUANT_mse

Algorithm 1 TURBOQUANT_{mse}: optimized for MSE

1

```
1: input: dimension  $d$  and bit-width  $b$   
   // Global Parameters for Setting up TURBOQUANTmse  
2: Generate a random rotation matrix  $\Pi \in \mathbb{R}^{d \times d}$   
3: Construct codebook by finding centroids  $c_1, c_2, \dots, c_{2^b} \in [-1, 1]$  that minimize MSE cost in Eq. (4)
```

2

```
4: Procedure QUANTmse( $x$ )  
5:  $y \leftarrow \Pi \cdot x$   
6:  $\text{idx}_j \leftarrow \arg \min_{k \in [2^b]} |y_j - c_k|$  for every  $j \in [d]$   $\{\text{idx}_j\text{'s are } b\text{-bit integers}\}$   
7: output:  $\text{idx}$ 
```

3

```
8: Procedure DEQUANTmse( $\text{idx}$ )  
9:  $\tilde{y}_j \leftarrow c_{\text{idx}_j}$  for every  $j \in [d]$   
10:  $\tilde{x} \leftarrow \Pi^\top \cdot \tilde{y}$   
11: output:  $\tilde{x}$ 
```

TurboQuant [Zandieh et al., arXiv 2025]

1

Setup

- Set dimension and bit-width
- Generate random rotation matrix Π
- Construct MSE-optimal codebook

2

Quantization

- $y \leftarrow \Pi \cdot x$
- Map each y_j to the nearest centroid index

3

Dequantization

- centroid lookup
- $\tilde{x} \leftarrow \Pi^\top \tilde{y}$

Step 1. Quantization

Quantization



b-bit index



Dequantization

1 Random Rotation

$$\rightarrow y \leftarrow \Pi \cdot x$$

- Make **random distribution** of KV Cache to **beta distribution**

2 Nearest Centroid Search

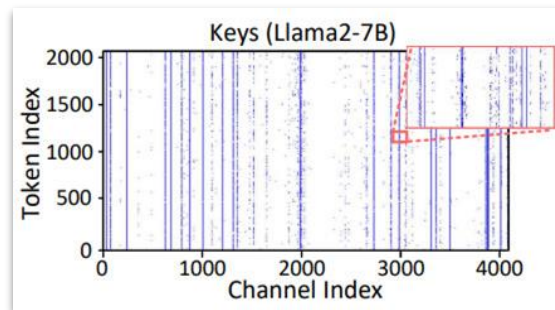
$$\rightarrow \text{idx}_j \leftarrow \text{argmin}_k |y_j - c_k|$$

- **Select the index** of the predefined centroid closest to y_j

3 Store the b-bit index

output: idx

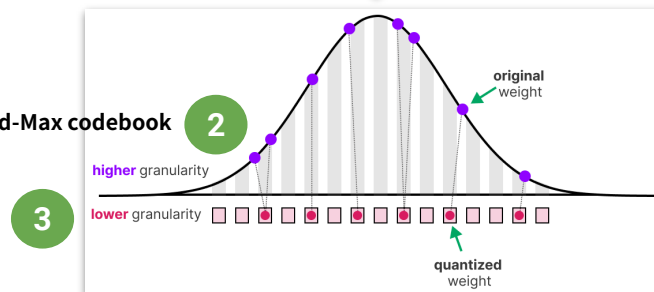
- store idx_j as sorted centroid (idx_j is b-bit integer)



Oaken [Kim et al., ISCA 2025]

↓ 1 $\pi(\frac{1}{\sqrt{d}} \times H \times D)$ projection

Lloyd-Max codebook



Step 2. Dequantization

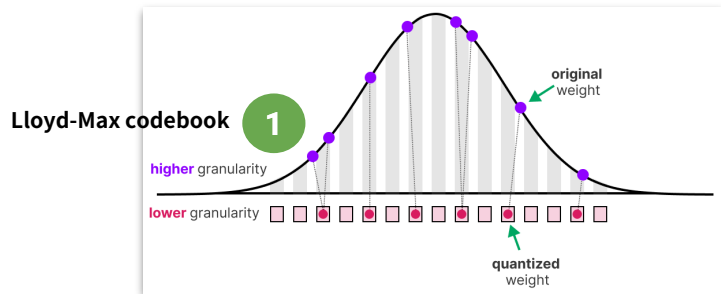
Quantization



b-bit index

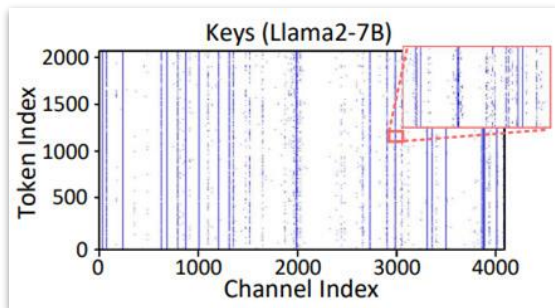


Dequantization



$$\pi^T (= \frac{1}{\sqrt{d}} \times D^T \times H^T) \text{ projection}$$

2



1 Centroid Lookup

$$\tilde{y}_j \leftarrow c_{idx_j}$$

- Use the index as address of the centroid LUT

- Read the corresponding centroid value to reconstruct $\tilde{y}_j$

2 Inverse Rotation

$$\rightarrow \tilde{x} \leftarrow \pi^T \cdot \tilde{y}$$

- Apply **inverse rotation** to return to the original basis

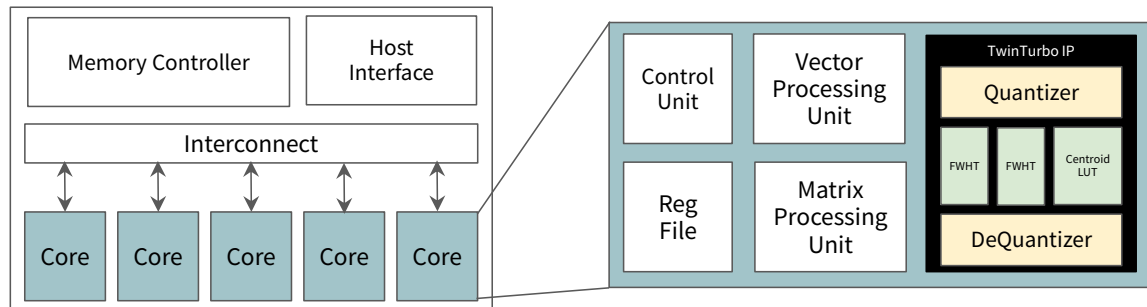
3 Output Approximation

output: $\tilde{x}$

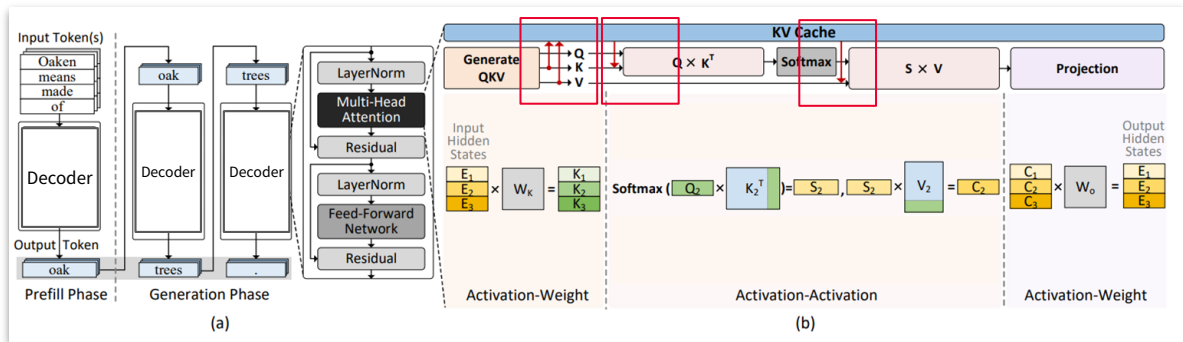
- Output the reconstructed approximation of x

3. HW Implementation

Overall Architecture



1 Resource optimization
: Optimized circuit resources by **sharing the FWHT module**, which is the critical path and primary operation.



2 Zero Architectural Changes Required
: **Applicable without altering the baseline accelerator architecture** by a Feeder IP integration structure.

Oaken [Kim et al., ISCA 2025]

Figure 2 : Overall TwinTurbo accelerator architecture.

Feeder IP Architecture

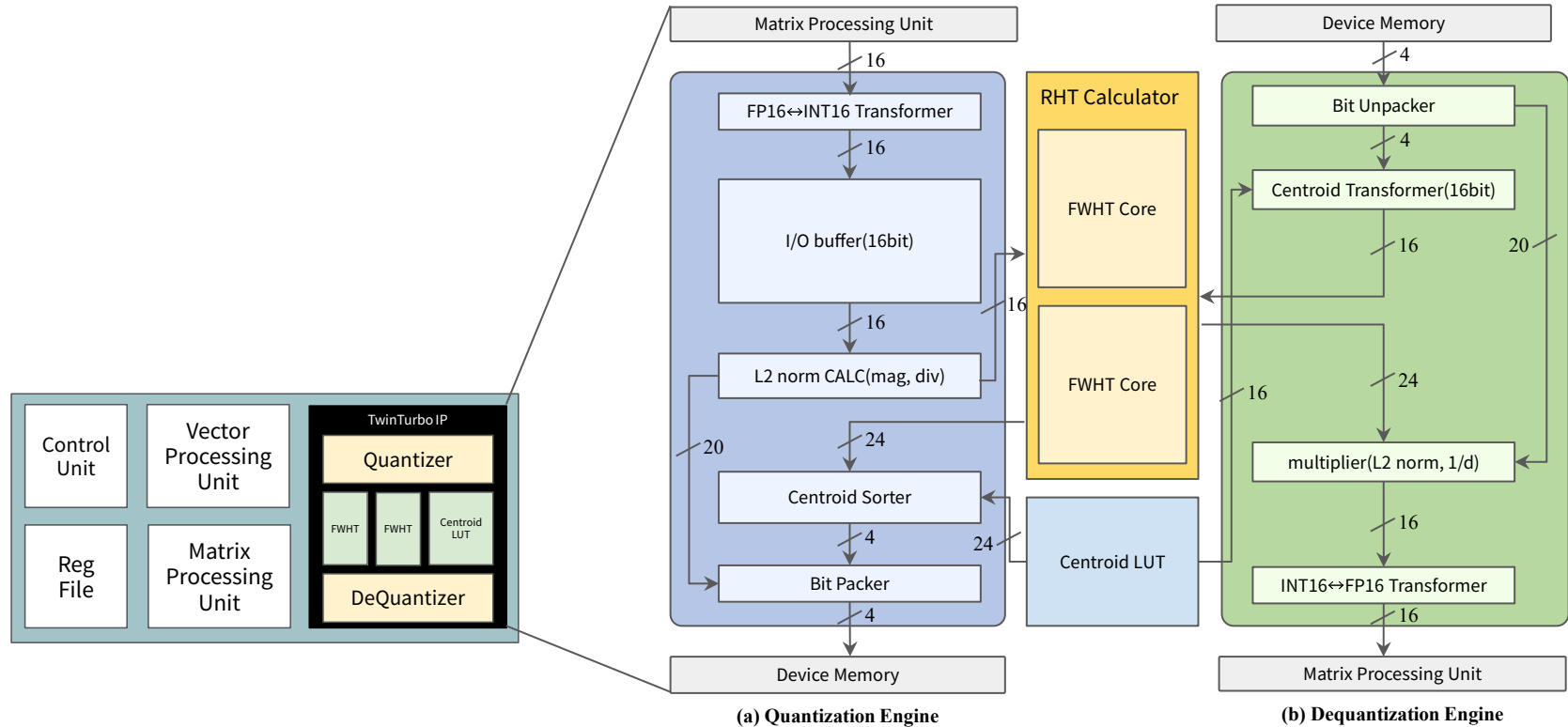


Figure 3 : Quantization and dequantization engines of TwinTurbo compute core.

RHT Calculator : Design Concepts

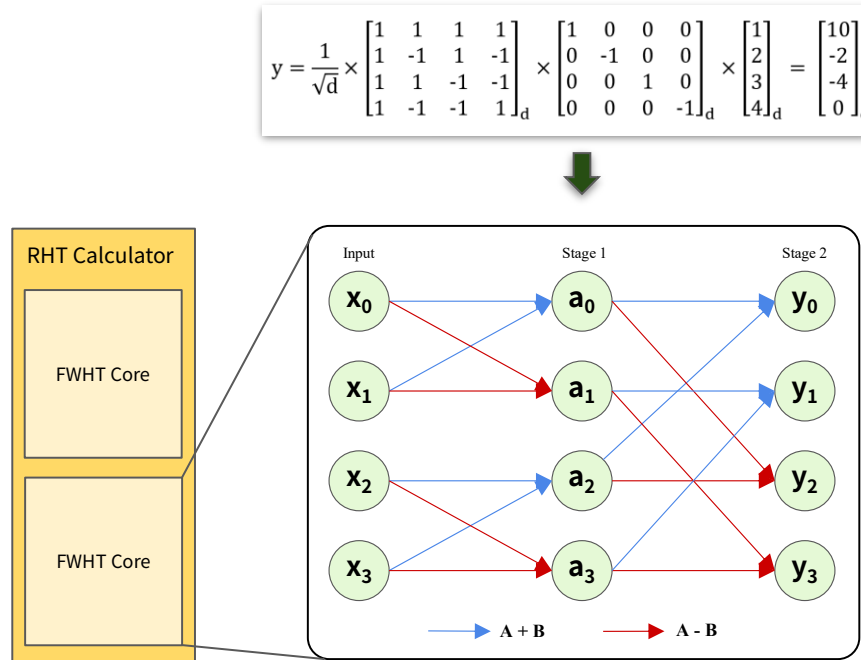
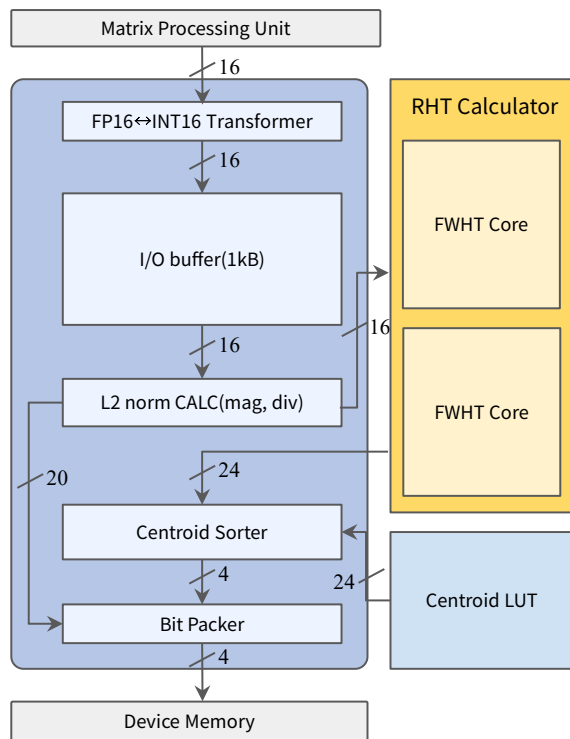


Figure 4 : TwinTurbo compute core.

- 1 **Resource optimized design**
: Replace Π operation to FWHT
(Fast Walsh-Hadamard Transform)
 $d^2 \rightarrow \log_2 D$
- 2 **Balance latency and hardware resource**
: Process 32 butterfly pairs in parallel
- 3 **Control the stage/phase**
: The module use an FSM-based control structure to repeat through Butterfly stages and phases.

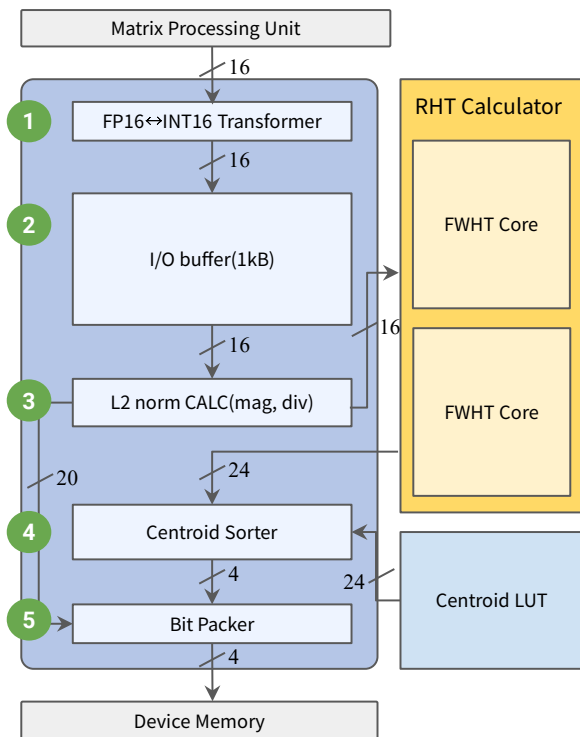
Quantization Engine : Design Concepts



(a) Quantization Engine

- 1 Resource optimized design**
: Trade-off between **resource** and **runtime**
→ Lane-based partial parallelism
- 2 Overflow-aware fixed-point datapath**
: Should consider
- **enough internal bit-width** for preventing overflow
- **saturation** for reducing computation load of next ops
- 3 Designed to Support Multiple Dimensions**
: To deal with multiple LLMs, TwinTurbo **should support multiple head dimensions**(64, 128, 256)

Quantization Engine : Dataflow & Modules



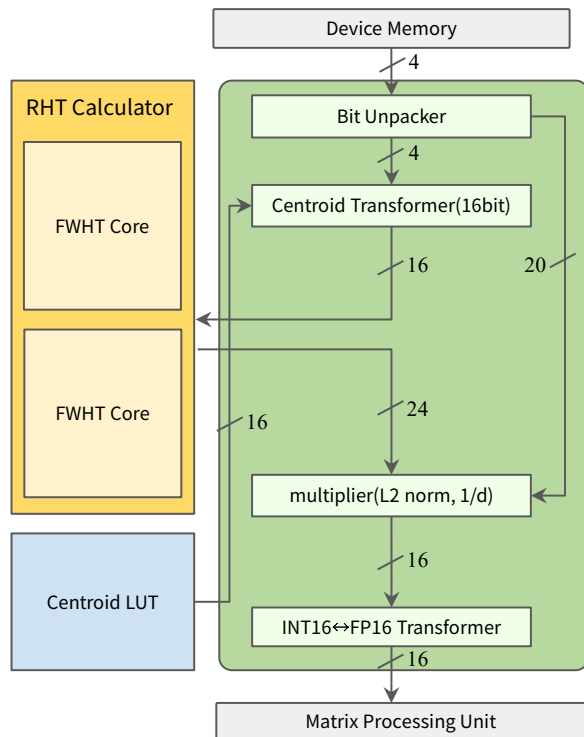
(a) Quantization Engine

- 1 **FP16 $\leftrightarrow$ INT16 Transformer**
: Converts FP16 K/V data into fixed-point format for internal computation
→ **32-lane / Total 2/4/8 cycles (d=64/128/256)**
- 2 **I/O buffer**
: Maximum d=256 * 16bits for each K & V Cache
→ **1kB storage capacity**
- 3 **L2 norm CALC**
: Normalize by multiplying the reciprocal of magnitude
→ **Total 66/71/79 cycles (d=64/128/256)**
- 4 **Centroid Sorter**
: Boundary comparator type 4bit centroid idx sorter (Thermometer → Binary idx transformation)
→ **8-lane / Total 9/17/33 cycles (d=64/128/256)**
- 5 **Bit Packer**
: Packing L2 norm magnitude & Quantized KV Caches

L2 norm magnitude(20bit)	Quantized KV Cache(4bit each)
--------------------------	-------------------------------

 → **Correspond to AXI4-Stream Interconnect (128bit)**

Dequantization Engine : Design Concepts



(b) Dequantization Engine

1 Cycle-critical Design

: Dequantization occurs at a rate that is the square of that of quantization.

→ Dequant needed to be FAST

2 Designed to Support Multiple Dimensions

: To deal with multiple LLMs, TwinTurbo **should support multiple head dimensions**(64, 128, 256)

Dequantization Engine : Dataflow & Modules

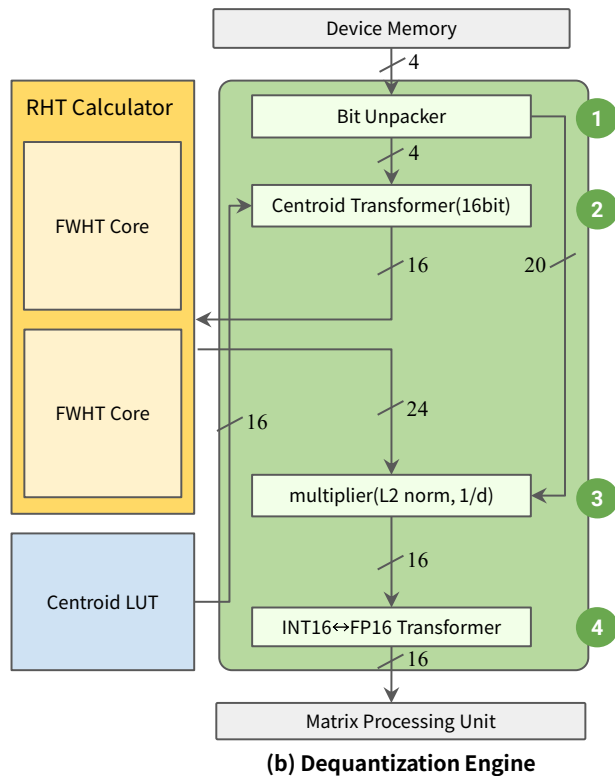


Figure 6 : Dequantization engines of TwinTurbo compute core.

1 Bit Unpacker

: Unacking L2 norm magnitude & Quantized KV Caches
→ Correspond to AXI4-Stream Interconnect (128bit)

2 Centroid Transformer

: Looks up 16-bit Lloyd-Max centroid values from a shared Centroid LUT using 4-bit quantized indices
→ 64-lane / Total 1/2/4 cycles (d=64/128/256)

3 Multiplier

: Scale it back by the magnitude and multiply by 1/d
→ 128-lane / Total 2/2/3 cycles (d=64/128/256)

4 INT16 to FP16 Transformer

: Converts dequantized fixed-values back to FP16 format for the NPU
→ 128-lane / Total 1/1/2 cycles (d=64/128/256)

💡 $y = \frac{1}{\sqrt{d}} \times H \times D \times x$ $\tilde{x} = \frac{1}{\sqrt{d}} \times D^T \times H^T \times y$

😊 $\tilde{x} = \frac{1}{d} \times D^T \times H^T \times H \times D \times x$ 😞

4. Results

Experimental setups

Item	Value
FPGA Board	Alveo U250 (XCU250)
Baseline Accelerator	llama-fpga (adamgallas/llama-fpga)
Target Model	TinyLlama-1.1B for d=64, Llama-2-7b for d=128, Gemma-2b for d=256
Feeder IP Precision	4-bit index, Q8.15/Q7.8 fixed-point
Head Dimension (d)	64 / 128 / 256 (TwinTurbo-NPU : d=128 only)
AXI Interface Width	128-bit (Feeder IP/AXI-4 Stream correspond)
Simulator	Vivado XSIM, ModelSim

1

Memory Efficiency through TwinTurbo

- **Standalone** Feeder IP **Compression Ratio Comparison**

2

TwinTurbo Quantization Quality

Inputs : **Actual KV cache data**

(TinyLlama-1.1B for d=64, Llama-2-7b for d=128, Gemma-2b for d=256 / 20~200 KV Caches from each model).

Evaluation : Compared against the **FP16 Python golden reference**.

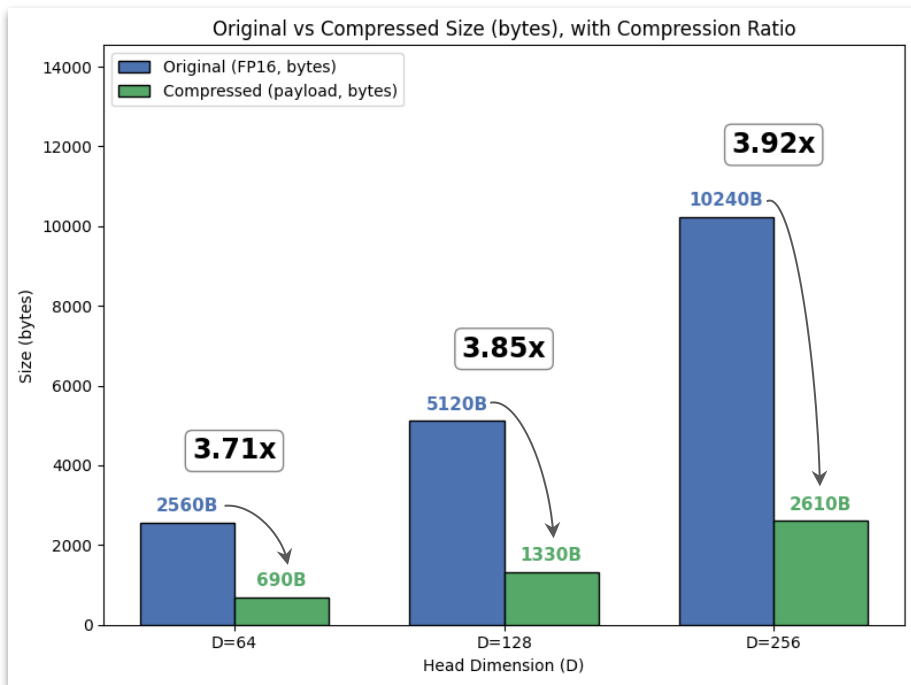
3

Full integration runtime analysis

- **Standalone** Feeder IP **Quant/Dequant Runtime Cycles**

Memory Efficiency & Quantization quality analysis

1



1

Compression Ratio

: Up to **3.92x compression efficiency** compared to the original FP16 KV cache input.
(Theoretically 4x compression efficiency)

2

Golden Reference vs. TwinTurbo Comparison

d=64,128 : **Very low** MSE rate (**0.88x, 1.24x of paper**).

d=256 : **Relatively high** MSE rate (**x111 of paper**).

Analysis : **due to increased fixed-point errors** as the dimension grows.

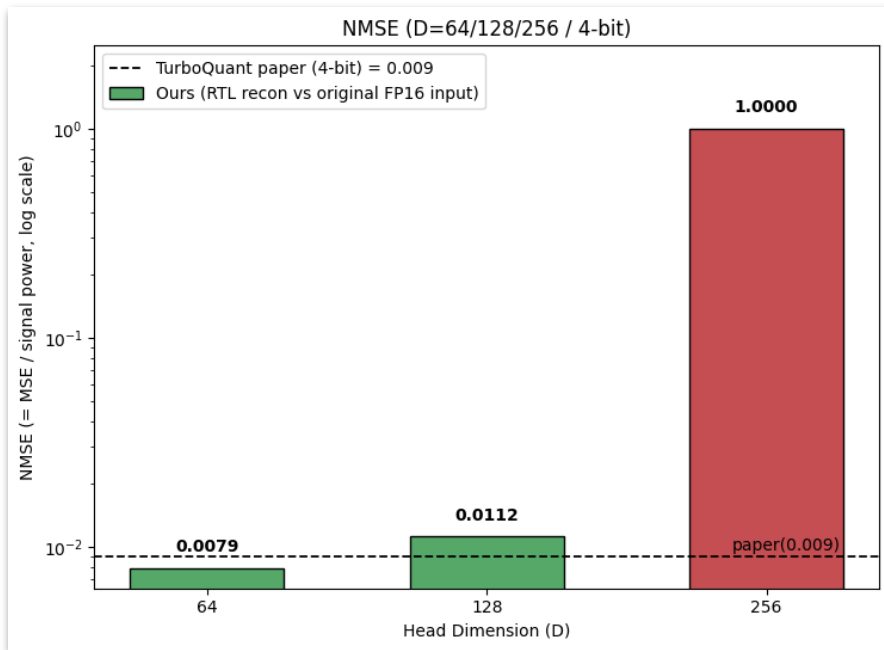
* TurboQuant Theoretical MSE value is 0.009



Inputs: Actual KV cache data (TinyLlama-1.1B for d=64, Llama-2-7b for d=128, Gemma-2b for d=256).
Evaluation: Compared against the FP16 Python golden reference.

Memory Efficiency & Quantization quality analysis

2



1

Compression Ratio

: Up to **3.92x compression efficiency** compared to the original FP16 KV cache input.
(Theoretically 4x compression efficiency)

2

Golden Reference vs. TwinTurbo Comparison

d=64,128 : **Very low** MSE rate (**0.88x, 1.24x of paper**).

d=256 : **Relatively high** MSE rate (**x111 of paper**).

Analysis : **due to increased fixed-point errors** as the dimension grows.

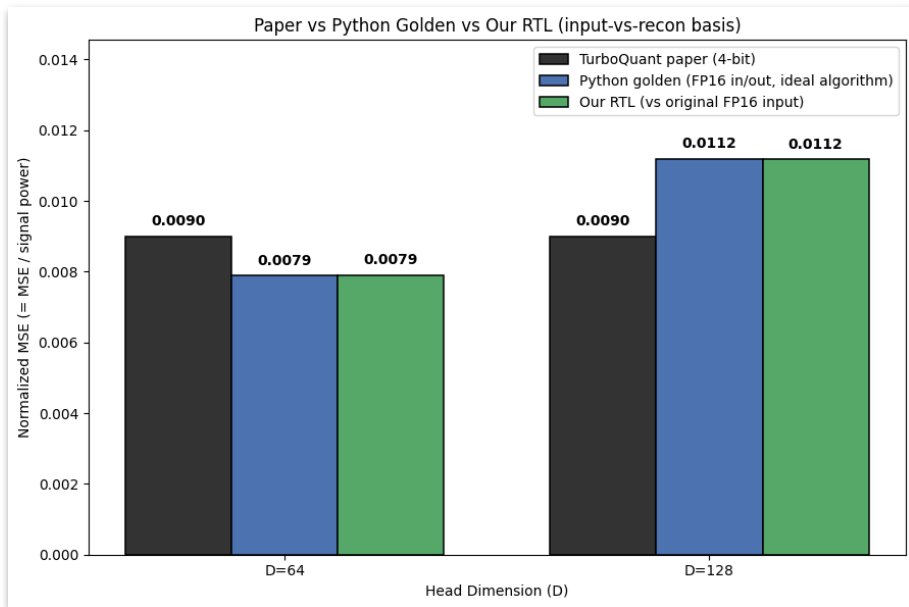
* TurboQuant Theoretical MSE value is 0.009



Inputs: Actual KV cache data (TinyLlama-1.1B for d=64, Llama-2-7b for d=128, Gemma-2b for d=256).
Evaluation: Compared against the FP16 Python golden reference.

Memory Efficiency & Quantization quality analysis

2



1

Compression Ratio

: Up to **3.92x compression efficiency** compared to the original FP16 KV cache input.
(Theoretically 4x compression efficiency)

2

Golden Reference vs. TwinTurbo Comparison

d=64,128 : **Very low** MSE rate (**0.88x, 1.24x of paper**).

d=256 : **Relatively high** MSE rate (**x111 of paper**).

Analysis : **due to increased fixed-point errors** as the dimension grows.

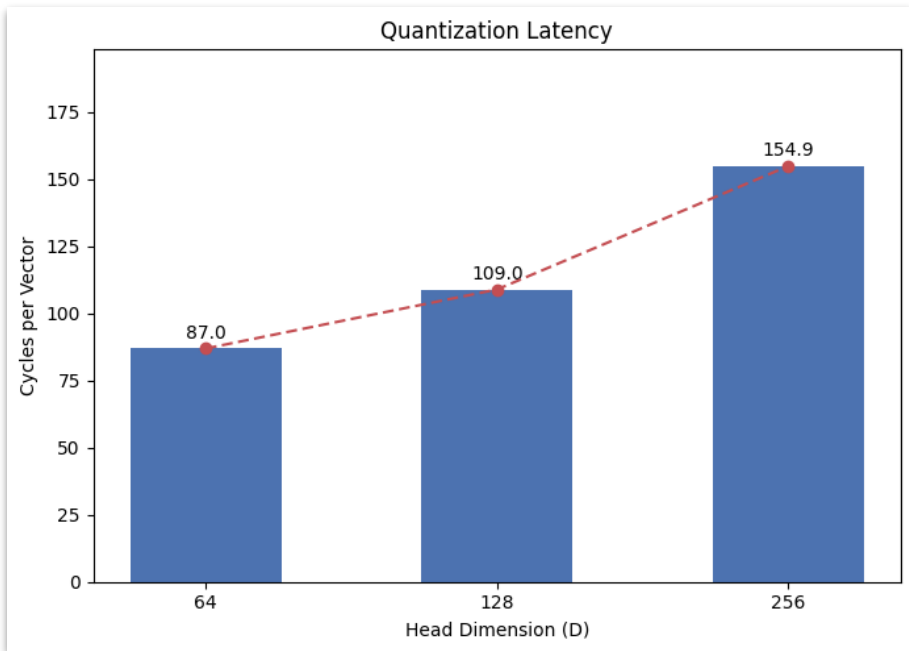
* TurboQuant Theoretical MSE value is 0.009



Inputs: Actual KV cache data (TinyLlama-1.1B for d=64, Llama-2-7b for d=128, Gemma-2b for d=256).
Evaluation: Compared against the FP16 Python golden reference.

Runtime analysis

1



1

Quantization Cycle Time

: Average of **87, 109, and 154 cycles per single head vector** for **d=64, 128, and 256**, respectively.

This overhead is negligible as it can be **latency hided during the decoding phase**.

2

Dequantization Cycle Time

: Average of **21, 33, and 61 cycles per single head vector** for **d=64, 128, and 256**, respectively.

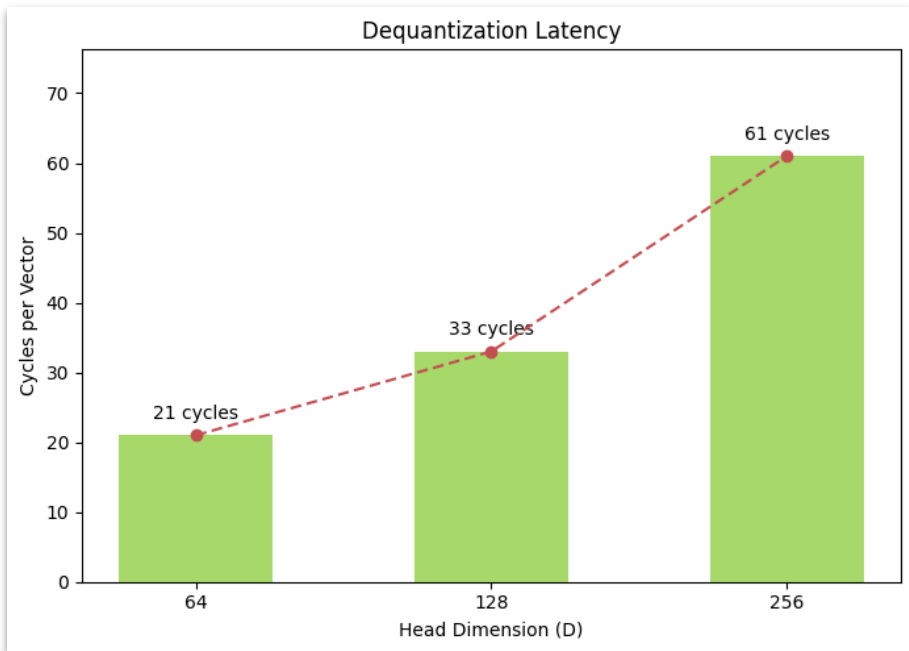
→ Relatively 2.5~4 times faster than Quantization



Inputs: Actual KV cache data (TinyLlama-1.1B for d=64, Llama-2-7b for d=128, Gemma-2b for d=256).
Evaluation: Compared against the FP16 Python golden reference.

Runtime analysis

2



1 Quantization Cycle Time

: Average of 87, 109, and 154 cycles per single head vector for $d=64$, 128, and 256, respectively.

This overhead is negligible as it can be latency hided during the decoding phase.

2 Dequantization Cycle Time

: Average of 21, 33, and 61 cycles per single head vector for $d=64$, 128, and 256, respectively.

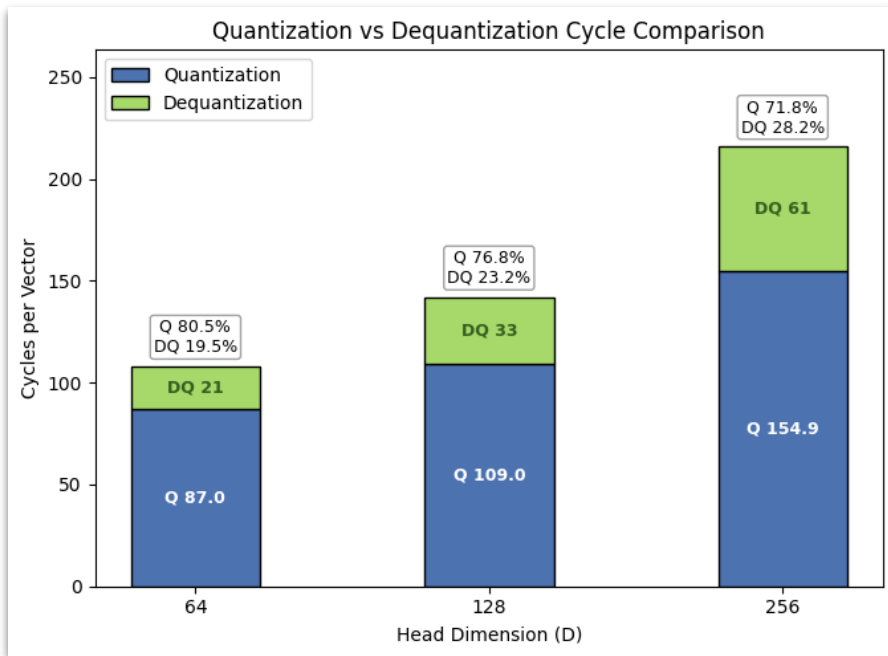
→ Relatively 2.5~4 times faster than Quantization



Inputs: Actual KV cache data (TinyLlama-1.1B for $d=64$, Llama-2-7b for $d=128$, Gemma-2b for $d=256$).
Evaluation: Compared against the FP16 Python golden reference.

Runtime analysis

2



1

Quantization Cycle Time

: Average of **87, 109, and 154 cycles per single head vector** for **d=64, 128, and 256**, respectively.

This overhead is negligible as it can be latency hided during the decoding phase.

2

Dequantization Cycle Time

: Average of **21, 33, and 61 cycles per single head vector** for **d=64, 128, and 256**, respectively.

→ Relatively 2.5~4 times faster than Quantization



Inputs: Actual KV cache data (TinyLlama-1.1B for d=64, Llama-2-7b for d=128, Gemma-2b for d=256).
Evaluation: Compared against the FP16 Python golden reference.

Conclusion & Future works

[Conclusion]

1 Near-Optimal Compression Ratio

: Achieved up to **3.85x** compression, closely approaching the 4x theoretical limit for exceptional memory efficiency.

2 High-Fidelity Quantization

: Delivered an **extremely low MSE**, achieving performance nearly identical to theoretical models and HLL implementations.

[Future works]

1 Dynamic Fixed-Point Q-Factor

: Support **adaptive Q-factor scaling** within the hardware datapath.

2 Non-Standard Dimension Support

: Extend architectural compatibility for **irregular head dimensions** (e.g., $d=80$).

Reference

- [1] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, "TurboQuant: Online vector quantization with near-optimal distortion rate," *arXiv preprint*, arXiv:2504.19874, Apr. 2025.
- [2] P. Chen and X. Yang, "Titanus: Enabling KV cache pruning and quantization on-the-fly for LLM acceleration," in *Proc. Great Lakes Symp. VLSI (GLSVLSI)*, New Orleans, LA, USA, Jun.–Jul. 2025, pp. 1–7.
- [3] M. Kim, S. Hong, R. Ko, S. Choi, H. Lee, J. Kim, J.-Y. Kim, and J. Park, "Oaken: Fast and efficient LLM serving with online-offline hybrid KV cache quantization," in *Proc. 52nd Annu. Int. Symp. Comput. Archit. (ISCA)*, Tokyo, Japan, Jun. 2025, pp. 1–16.
- [4] R. Ben-Basat, W. Kuszmaul, M. Mitzenmacher, A. Portnoy, and S. Vargafik, "Quantizing with randomized Hadamard transforms: Efficient heuristic now proven," *arXiv preprint*, arXiv:2605.06014, May 2026.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.
- [6] D. Lemire and L. Boytsov, "Decoding billions of integers per second through vectorization," *Softw., Pract. Exper.*, vol. 45, no. 1, pp. 1–29, 2015.

THANK YOU

Q & A

APPENDIX - Algorithm1 : Mathematical approach

1 Generate Random Rotation Matrix Π

- $\Pi \in \mathbb{R}^{d \times d}$
- **Random Rotation: Evenly distributes** component values.
- **Worst-case Mitigation : Prevents value concentration** on specific coordinates (outliers).



Simplify Quantization as indexing

2 Generate MSE-optimal Codebook

- **Component Range:** $[-1, 1]$
- **Component Distribution: Beta Distribution**
- **Approximates Normal Distribution** in high dimensions.
- **Centroid Calculation: Computes 2^b centroids** based on this distribution.

$$\mathcal{C}(f_X, b) := \min_{-1 \leq c_1 \leq c_2 \leq \dots \leq c_{2^b} \leq 1} \sum_{i=1}^{2^b} \int_{\frac{c_{i-1} + c_i}{2}}^{\frac{c_i + c_{i+1}}{2}} |x - c_i|^2 \cdot f_X(x) \, dx.$$



Before actual quantization, process of predetermine centroid~idx bits

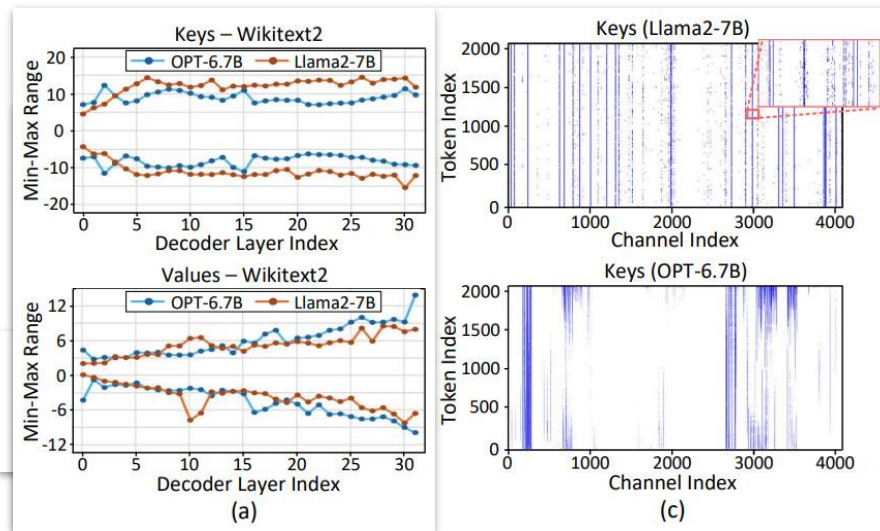
APPENDIX - Algorithm1 : Mathematical approach

1 Generate Random Rotation Matrix Π

- $\Pi \in \mathbb{R}^{d \times d}$
- **Random Rotation: Evenly distributes** component values.
- **Worst-case Mitigation : Prevents value concentration** on specific coordinates (outliers).

2 Generate MSE-optimal Codebook

- **Component Range:** $[-1, 1]$
- **Component Distribution: Beta Distribution**
- **Approximates Normal Distribution** in high dimensions.
- **Centroid Calculation: Computes 2^b centroids** based on this distribution.



Oaken [Kim et al., ISCA 2025]



Before actual quantization, process of predine centroid~idx bits

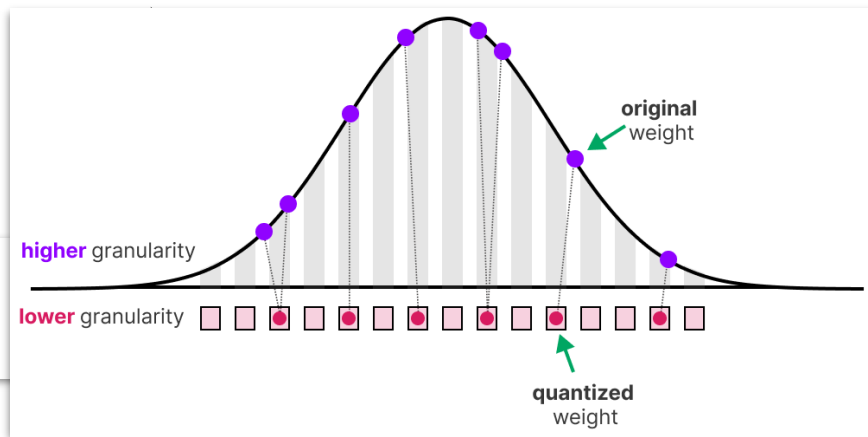
APPENDIX - Algorithm1 : Mathematical approach

1 Generate Random Rotation Matrix Π

- $\Pi \in \mathbb{R}^{d \times d}$
- **Random Rotation: Evenly distributes** component values.
- **Worst-case Mitigation : Prevents value concentration** on specific coordinates (outliers).

2 Generate MSE-optimal Codebook

- **Component Range:** $[-1, 1]$
- **Component Distribution: Beta Distribution**
- **Approximates Normal Distribution** in high dimensions.
- **Centroid Calculation: Computes 2^b centroids** based on this distribution.



Before actual quantization, process of predine centroid~idx bits

APPENDIX - Algorithm2 : TURBOQUANT_prod

1

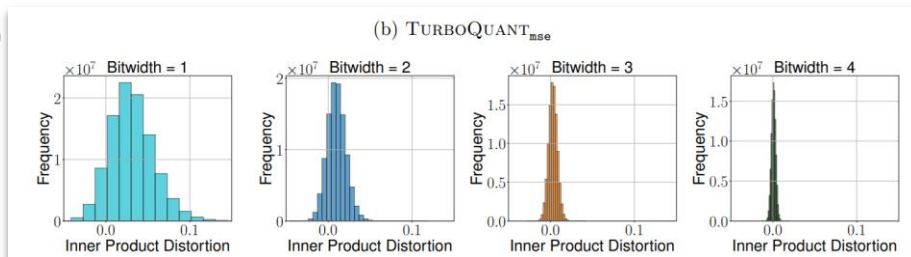


Figure 1: Error distribution of TURBOQUANT_{prod} and TURBOQUANT_{mse} for Inner Product Estimation.

$$\mathbb{E} [\langle \mathbf{y}, Q_{\text{mse}}^{-1}(Q_{\text{mse}}(\mathbf{x})) \rangle] = \frac{2}{\pi} \cdot \langle \mathbf{y}, \mathbf{x} \rangle$$

TurboQuant [Zandieh et al., arXiv 2025]

2

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

1

Bias Issue in Algorithm 1

: MSE-based loss optimization mathematically induces a **$2/\pi$ multiplicative bias**.

2

Mitigating Exponential Amplification in Softmax

: Multiplicative bias combined with softmax leads to exponential amplification, **risking errors in attention probability distributions**.

3

Introduction of QJL

: Aims to ensure **unbiasedness with minimal bit usage**.

APPENDIX - Algorithm2 : TURBOQUANT_prod

Algorithm 2 TURBOQUANT_{prod}: optimized for inner product

```

1: input: dimension  $d$  and bit-width  $b$ 
   // Global Parameters for Setting up TURBOQUANTprod
2: Instantiate a TURBOQUANTmse with bit-width  $b - 1$  as per Algorithm 1
3: Generate a random projection matrix  $S \in \mathbb{R}^{d \times d}$  with i.i.d. entries  $S_{i,j} \sim \mathcal{N}(0, 1)$ 

4: Procedure QUANTprod( $x$ )
5:  $\text{idx} \leftarrow \text{QUANT}_{\text{mse}}(x)$ 
6:  $r \leftarrow x - \text{DEQUANT}_{\text{mse}}(\text{idx})$  {residual vector}
7:  $\text{qjl} \leftarrow \text{sign}(S \cdot r)$  {QJL on residual vector}
8: output: ( $\text{idx}, \text{qjl}, \|r\|_2$ )

9: Procedure DEQUANTprod( $\text{idx}, \text{qjl}, \gamma$ )
10:  $\tilde{x}_{\text{mse}} \leftarrow \text{DEQUANT}_{\text{mse}}(\text{idx})$ 
11:  $\tilde{x}_{\text{qjl}} \leftarrow \frac{\sqrt{\pi/2}}{d} \cdot \gamma \cdot S^T \cdot \text{qjl}$ 
12: output:  $\tilde{x}_{\text{mse}} + \tilde{x}_{\text{qjl}}$ 
    
```

TurboQuant [Zandieh et al., arXiv 2025]

1 Residual Calculation

: Generates a residual vector (r) between the original and dequantized vectors.
(Also computes L2 norm of r)

2 QJL Operation - Matrix S

: Generates a random matrix $S \sim \mathcal{N}(0, 1)$

3 QJL Ops - Signum Funct

: Generates 1-bit QJL by passing dot product of the residual vector and matrix S through a signum function.

4 Final Bit Packaging

APPENDIX - Why only Algorithm 1?

Algorithm 2 $\text{TURBOQUANT}_{\text{prod}}$: optimized for inner product

```
9: Procedure  $\text{DEQUANT}_{\text{prod}}(\text{idx}, \text{qjl}, \gamma)$   
10:  $\tilde{x}_{\text{mse}} \leftarrow \text{DEQUANT}_{\text{mse}}(\text{idx})$   
11:  $\tilde{x}_{\text{qjl}} \leftarrow \frac{\sqrt{\pi/2}}{d} \cdot \gamma \cdot S^T \cdot \text{qjl}$   
12: output:  $\tilde{x}_{\text{mse}} + \tilde{x}_{\text{qjl}}$ 
```

TurboQuant [Zandieh et al., arXiv 2025]

The Surprising Part

QJL actually hurts performance in practice.

Here's what I found on Qwen3-1.7B (4K context), top-1 token consistency rate drops:

Bits	MSE-only Top1%	+QJL Top1%	Difference
2	65.6%	50.0%	-15.6%
3	71.0%	61.2%	-9.8%
4	80.4%	69.6%	-10.7%
6	94.2%	89.7%	-4.5%
8	96.9%	94.6%	-2.2%

MSE-only consistently wins on Top-1 token matching. The gap is huge at low bits and still noticeable at 8-bit.

1

Variance Increase via QJL

: Quantizing the residual amplifies noise at the individual token level.

2

Hardware Overhead in Operations

: Unlike the Π matrix, matrix S uses $N(0, 1)$ random elements, raising concerns about massive runtime overhead.

3

Importance of Dequantization Optimization

: Masking dequantization latency from DRAM reads is crucial in Multi-Head Attention, making Algorithm 1 more advantageous.

APPENDIX - Why only Algorithm 1?

Algorithm 2 $\text{TURBOQUANT}_{\text{prod}}$: optimized for inner product

```
9: Procedure  $\text{DEQUANT}_{\text{prod}}(\text{idx}, \text{qj1}, \gamma)$   
10:  $\hat{x}_{\text{mse}} \leftarrow \text{DEQUANT}_{\text{mse}}(\text{idx})$   
11:  $\hat{x}_{\text{qj1}} \leftarrow \frac{\sqrt{\pi/2}}{d} \cdot \gamma \cdot S^T \cdot \text{qj1}$   
12: output:  $\hat{x}_{\text{mse}} + \hat{x}_{\text{qj1}}$ 
```

TurboQuant [Zandieh et al., arXiv 2025]

1 Variance Increase via QJL

: Quantizing the residual amplifies noise at the individual token level.

2 Hardware Overhead in Operations

: Unlike the Π matrix, matrix S uses $N(0, 1)$ random elements, raising concerns about massive runtime overhead.

3 Importance of Dequantization Optimization

: Masking dequantization latency from DRAM reads is crucial in Multi-Head Attention, making Algorithm 1 more advantageous.

What's Going On?

The theory says QJL = no bias. That's true! But here's the trade-off:

Bits	Bias Improvement	Variance Increase
2	-11.65%	+17.2%
3	-2.83%	+30.7%
4	-0.88%	+329.8%
6	-0.02%	+368.6%

QJL eliminates bias but explodes variance. And for attention, variance is worse than bias!

Why? Softmax is tolerant to uniform bias:

$$\text{softmax}(\text{scores} + \text{constant_bias}) \approx \text{softmax}(\text{scores})$$

But variance randomly perturbs each score, which messes up Top-K ranking:

$$\text{argmax}(\text{scores} + \text{noise}) \neq \text{argmax}(\text{scores})$$

So you get "unbiased" estimates that give you the wrong Top-1 token more often.

APPENDIX - Why only Algorithm 1?

Algorithm 1 $\text{TURBOQUANT}_{\text{mse}}$: optimized for MSE

```
8: Procedure  $\text{DEQUANT}_{\text{mse}}(\text{idx})$   
9:  $\tilde{\mathbf{y}}_j \leftarrow c_{\text{idx}_j}$  for every  $j \in [d]$   
10:  $\tilde{\mathbf{x}} \leftarrow \Pi^T \cdot \tilde{\mathbf{y}}$   
11: output:  $\tilde{\mathbf{x}}$ 
```

Algorithm 2 $\text{TURBOQUANT}_{\text{prod}}$: optimized for inner product

```
9: Procedure  $\text{DEQUANT}_{\text{prod}}(\text{idx}, \text{qj1}, \gamma)$   
10:  $\tilde{\mathbf{x}}_{\text{mse}} \leftarrow \text{DEQUANT}_{\text{mse}}(\text{idx})$   
11:  $\tilde{\mathbf{x}}_{\text{qj1}} \leftarrow \frac{\sqrt{\pi/2}}{d} \cdot \gamma \cdot \mathbf{S}^T \cdot \text{qj1}$   
12: output:  $\tilde{\mathbf{x}}_{\text{mse}} + \tilde{\mathbf{x}}_{\text{qj1}}$ 
```

TurboQuant [Zandieh et al., arXiv 2025]

1 Variance Increase via QJL

: Quantizing the residual amplifies noise at the individual token level.

2 Hardware Overhead in Operations

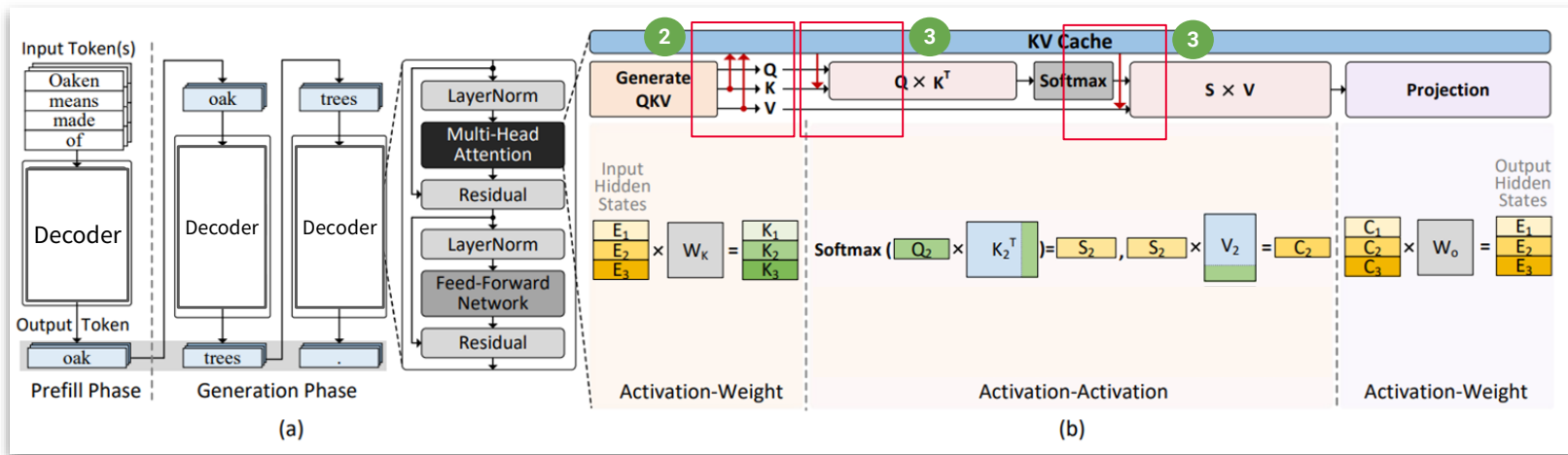
: Unlike the Π matrix, matrix $\mathbf{S}$ uses $\mathcal{N}(0, 1)$ random elements, raising concerns about massive runtime overhead.

3 Importance of Dequantization Optimization

: Masking dequantization latency from DRAM reads is crucial in Multi-Head Attention, making Algorithm 1 more advantageous.

→ **Simplified Dequantization is crucial!**

APPENDIX - LLM Inference explanation



Oaken [Kim et al., ISCA 2025]

1 Token Embedding, QKV Projection

"맑아요" → [2345] → [0.2, 0.7, ...] ($1 \times d$ 벡터)

X_T ($1 \times d$)
 $\downarrow$
 $Q_T = X_T \cdot W_Q$ ($1 \times d$) ← 새 Query
 $K_T = X_T \cdot W_K$ ($1 \times d$) ← 새 Key
 $V_T = X_T \cdot W_V$ ($1 \times d$) ← 새 Value

2 KV Cache Update

: New K_T, V_T → Stored additionally on DRAM
 (K_T will be added on past $K_1 \sim K_{\{N\}}$)

→ In this process, Quantization of KV Cache will be proceeded
 (Quantization Latency will be latency-hided by Attention operations)

3 Attention Operations

: Read past $K_1 \sim K_{\{N+T\}}$ from DRAM
 → In this process, Dequantization of Compressed KV Cache will be proceeded

$scores = \langle \Pi \cdot Q_T, y_{\sim K} \rangle / \sqrt{d}$ ($1 \times N+T$ 벡터)
 $weights = softmax(scores)$
 $output = weights \cdot y_{\sim V}$